

## Unit 2

Methods to find  $\pi^*$

- Policy-based - training the policy directly
- Value-based - train to learn which state is more valuable & the action that leads to it.

In value-based methods, a value function is learned. The value is the expected discounted return if the agent starts at that state and acts according to the policy.

$$V_{\pi}(s) = E_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s]$$

Value-based methods have no policy  $\rightarrow$  policy is hard-coded ex. always choose the action w/ the highest value (greedy policy)

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} Q^*(s, a)$$

most of the time an epsilon-greedy policy is used

State Value Function

$$V_{\pi}(s) = E_{\pi} [G_t | S_t = s]$$

Value

expected return

if the agent starts at s.

given that you're following the policy

Action Value Function

$$Q_{\pi}(s, a) = E_{\pi} [G_t | S_t = s, A_t = a]$$

Value of a state-action pair

expected return given policy  $\pi$

starting at state s

picking action a.

## The Bellman Equation

$$V_{\pi}(s) = E_{\pi} [R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

Value of state  $s$       Expected value of immediate reward      + discounted value of the next state      given a starting state of  $s$ .

Monte Carlo & Temporal Difference Learning are strategies on how to train the value & policy.

Monte Carlo uses a whole episode of experience. Temporal difference uses only one step to learn.

Monte Carlo waits until the end of an episode, calculates  $G_t$  (return) & uses it as a target while updating  $V(S_t)$ . Requires a complete episode of interaction before learning.

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

new value of state  $t$       former estimation of value      learning rate      return at timestep  $t$       former estimation of value

Temporal Difference Learning: waits for only 1 interaction, update at each step!

Updates based on estimate, not complete  $G_t$  (bootstrapping)

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

new value of state  $t$       for estimate      learning rate      reward      discounted value of next state      former estimate

ID target rather than  $G_t$

**Q Learning** is an off-policy, value-based method, that uses a TD approach to train its action-value function.

**Q function** - an action-value function that determines the value of being at a particular state & taking a specific action at that state. The Q comes from the Quality (value) of that action at that state.

↳ Encoded by a **Q table**, where each cell corresponds to state-action pair value. (cheat sheet)

→ **Q learning algorithm**

**Step 1.** initialize Q arbitrarily:  $Q(s, a) = 0$  for all  $s \in S$  &  $a \in A(s)$ , &  $Q(\text{terminal state}, \cdot) = 0$

States      Actions      → all zeros. Q table.

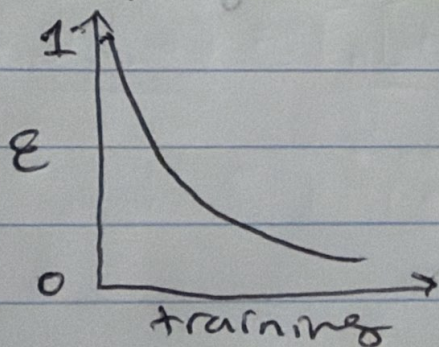
|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

**Step 2** choose an action using the epsilon greedy strategy

$\epsilon$ -greedy policy

- $1 - \epsilon$  → Exploitation (selects the greedy action)
- $\epsilon$  → Exploration (selects the random action)

↳ using an initial value of  $\epsilon = 1$   
w/ probability  $1 - \epsilon(0) \rightarrow$  do exploitation  
w/ prob.  $\epsilon(1) \rightarrow$  do exploration.



as the Q function improves,  $\epsilon$  can be decreased as exploration is less necessary.



→

### Step 3

perform action  $A_t$ , observe  $R_{t+1}$  &  $S_{t+1}$

### Step 4 - update $Q(S_t, A_t)$

using TD learning, update our value function after 1 step of interaction.

to produce the TD target, use the immediate new.  $R_{t+1}$  + discounted val at next state.

$Q(S_t, A_t)$  update formula

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New Q value estimation

former estimate

l.r reward

discounted estimate of optimal Q value at next state

TD target: obtain  $R_{t+1}$

after taking action  $A_t$

then use a greedy policy ( $\arg\max$ ) to select the estimate the reward of the next state.

↳ Off-policy: means that it uses one policy (e-greedy) to actually take actions, but uses another policy (greedy) to estimate rewards

off policy - uses different policies for acting & updating, on-policy uses the same.

